

抽象的

多模态大语言模型 (MLLM) 可以实现强大的跨模态推理, 但会带来巨大的计算和延迟负担, 这给资源受限环境中的部署带来了严峻的挑战。在本文中, 我们提出了 MoA-Off, 这是一种自适应异构模态感知卸载框架, 具有边云协作功能, 可实现高效的 MLLM 推理。MoA-Off 引入了一种轻量级异构模态感知模块, 该模块通过多维特征分析来估计异构输入的复杂性。然后, 提出了一种自适应边缘-云协作卸载策略, 该策略根据模态感知的复杂度分数和实时系统状态动态调度边缘和云之间的工作负载。实验结果表明, 与传统方法相比, MoA-Off 可以实现延迟降低 30% 以上, 资源开销降低 30%–65%, 同时保持有竞争力的准确性。

Index Terms— 多模态大语言模型、边云协作、自适应卸载、推理优化

1. 简介

多模态大语言模型 (MLLM) 最近在集成视觉、听觉和文本信息方面表现出了卓越的能力, 从而实现了跨异构数据源的统一推理[1]。如图 1 所示。这些模型越来越多地部署在从智能助手到实时感知系统等应用中 [2]。然而, 它们令人印象深刻的性能是以大量计算和内存需求为代价的[3], 这在延迟敏感和资源受限的环境中提出了严峻的挑战[4]。纯基于云的解决方案虽然功能强大, 但通常会遭受过大的传输延迟、带宽开销和潜在的隐私风险[5]。相比之下, 仅边缘方法提供更短的响应时间和更好的局部性, 但从根本上受到计算能力有限的限制[6], 导致复杂多模态任务的性能下降[7]。

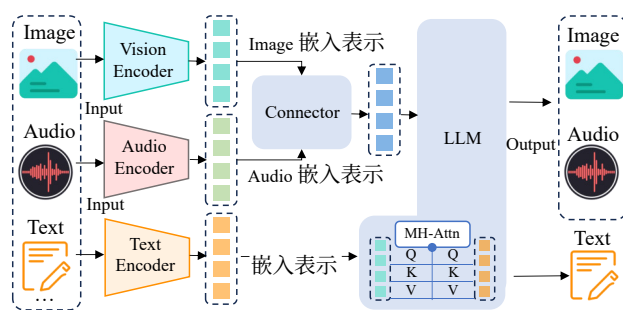


图 1. MLLM 的示例。

为了解决这些限制, 最近的研究探索了边缘-云协作推理[8, 9], 其中在边缘设备上进行轻量级预处理, 并将计算密集型推理卸载到云端。此类混合解决方案通过跨异构资源分配工作负载来平衡效率和准确性。然而, 现有的协作框架通常采用统一的卸载策略, 处理多模式输入而不考虑其内在的异质性[11]。在实践中, 不同的模态表现出不同的计算特征、通信成本和对推理准确性的敏感性[12]。忽略这些差异通常会导致任务分配不理想、资源利用效率低下以及用户体验不一致[13]。

在本文中, 我们提出了 MoA-Off, 这是一种自适应模态感知卸载框架, 用于通过边云协作进行高效 MLLM 推理。我们的方法将特定于模态的复杂性引入卸载过程, 从而提高系统效率和推理性能。主要贡献如下:

- 我们开发了一个轻量级的异构模态感知模块, 它通过有效的特征提取来估计图像和文本输入的复杂性, 同时保持边缘设备的最小开销。
- 我们提出了一种自适应边缘云协作-

卸货政策。该策略可实现细粒度调度，确保将资源密集型模式卸载到云端以进行准确推理，同时将轻量级输入保留在边缘以进行低延迟处理。

- 我们对多模式基准数据集进行了广泛的实验。实验结果表明，与传统方法相比，MoA-Off 在保持有竞争力的准确性的同时，延迟时间减少了 30% 以上，同时资源开销减少了 30%–65%。

2. 准备工作

多模态大语言模型。MLLM 通过将文本、图像、音频和视频等异构模式集成到统一的推理框架中来扩展传统的基于文本的 LLM [14]。形式上，给定对应于文本、视觉和音频的模式输入集 x_t, x_v, x_a ，模型通过相应的编码器 $f_m(\cdot)$ 构造模式嵌入 $h_m = f_m(x_m)$ ，然后通过投影函数 $g(\cdot)$ 将它们对齐到公共潜在空间：

$$z_m = g(h_m), \quad m \in \{t, v, a\}. \quad (1)$$

这种表述使 MLLM 能够处理复杂的任务，例如多模态问答、字幕和基础推理，但也会导致大量的计算和内存需求，使高效部署成为一个不小的挑战。

边缘云架构。边缘云架构提供了一种分层计算范式，可以在资源受限的边缘设备和强大的云服务器之间分配推理工作负载，从而平衡延迟、能源效率和计算能力[?]。这种划分可以实现低延迟响应，同时利用云的可扩展性进行复杂推理[15]。与纯粹的集中式解决方案相比，这种协作设计减轻了独立边缘推理的局限性，同时减少了对云的依赖，使其特别适合动态网络和资源条件下的实时多模态应用。

3. 所提出的方法

我们提出了 MoA-Off，这是一种自适应异构模式感知卸载框架，旨在通过边云协作来提高 MLLM 推理的效率，如图 2 所示。它主要由两部分组成：(1)

Lightweight Heterogeneous Modality-Aware 和 (2) *Adaptive Edge-Cloud Collaborative Offloading* 卸载。MoA-Off 的核心思想是利用不同模式的异构特征，在边缘和云之间动态分配计算工作负载。

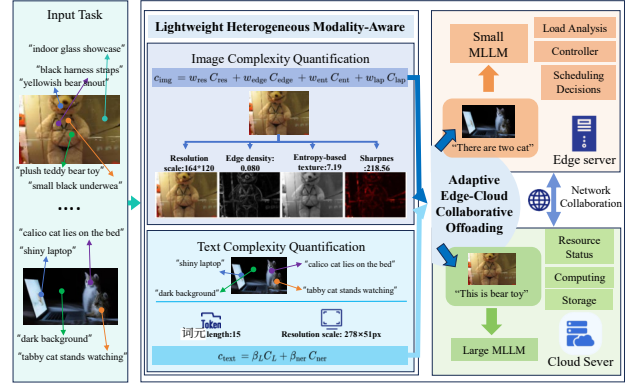


图 2. 拟议的 MoA-Off 框架概述。

3.1. 轻量级异构模式感知

轻量级异构模式感知模块旨在有效识别边缘设备上每种输入模式的类型并估计其复杂性。其主要目标是提供轻量级但可靠的复杂性分数，作为自适应卸载策略的基础。为了确保实用性，所有指标均源自快速、单通道特征提取技术[16]，使其适合在资源有限的环境中实时部署。

3.1.1. Image Complexity

对于具有 $N = HW$ 像素的输入图像 $I \in [0, 255]^{H \times W}$ ，图像复杂性被量化为捕获视觉信息不同方面的几个轻量级指标的加权组合。具体来说，我们考虑分辨率尺度、边缘密度、基于熵的纹理和清晰度。总体复杂度计算为加权和：

$C_{img} = w_{res}C_{res} + w_{edge}C_{edge} + w_{ent}C_{ent} + w_{lap}C_{lap}$ ，其中权重 w 为非负值并归一化为总和为 1。各指标定义如下。

分辨率比例。更高分辨率的图像通常需要更多的计算。我们相对于参考分辨率 (H_0, W_0) 标准化图像大小：

$$C_{res} = \min \left(1, \frac{HW}{H_0W_0} \right).$$

边缘密度。边缘密集的图像通常包含更丰富的结构信息，因此处理成本更高。我们测量平均索贝尔梯度大小并将其标准化：

$$C_{edge} = \text{clip} \left(\frac{\bar{G} - P_5(G)}{P_{95}(G) - P_5(G) + \epsilon}, 0, 1 \right). \quad (2)$$

基于熵的纹理。纹理丰富度通过从灰度直方图导出的图像熵来近似：

$$H(I) = - \sum_{k=0}^{255} p_k \log p_k, \quad C_{ent} = \frac{H(I)}{\log 256}, \quad (3)$$

其中 p_k 是灰度级 k 的归一化频率。

锐度。清晰或详细的图像处理起来要求更高。我们使用拉普拉斯算子的方差作为锐度指标：

$$C_{\text{lap}} = \text{clip} \left(\frac{\text{Var}(\nabla^2 I) - P_5}{P_{95} - P_5 + \epsilon}, 0, 1 \right), \quad (4)$$

其中 P_5 和 P_{95} 表示校准集中拉普拉斯方差的第 5 个和第 95 个百分位数。

3.1.2. Text Complexity

文本复杂性反映了长序列、实体密度和上下文依赖性引入的处理开销[17]。最终文本复杂性得分计算如下：

$C_{\text{text}} = \beta_L C_L + \beta_{\text{ner}} C_{\text{ner}}$ ，其中 β_L 和 β_{ner} 是权重。

令牌长度。较长的序列会导致变压器计算量呈二次方增长。我们相对于阈值 L_0 标准化标记计数 L ： $C_L = \min \left(1, \frac{L}{L_0} \right)$ 。

实体或数字密度。命名实体、数字表达式或专门术语的存在通常会增加语义推理的复杂性。我们通过计算每个句子的平均实体数量来衡量这个因素： $C_{\text{ner}} = \min \left(1, \frac{E/S}{\gamma} \right)$ ，其中 E 是实体数量， S 是句子数量， γ 是缩放常数。

3.2. 自适应边缘-云协作卸载

基于 Section 3.1 中引入的特定于模式的复杂性估计，我们设计了一种自适应卸载策略，该策略动态确定每种模式是否应在边缘本地处理或卸载到云端。主要目标是最大限度地减少延迟和资源开销，同时保持推理准确性。

对于每个模式 $m_i \in \text{text}$, 图像, 令 $c_i \in [0, 1]$ 表示模式感知模块的估计复杂度得分。系统状态由 $s = (\ell, b)$ 表示，其中 ℓ 是当前边缘负载， b 是可用网络带宽。卸载决策 d_i 定义为：

$$d_i = \begin{cases} \text{edge}, & c_i \leq \tau_{m_i} \wedge \ell \leq \ell_{\max} \wedge b \leq \beta, \\ \text{cloud}, & \text{otherwise}, \end{cases} \quad (5)$$

其中 τ_{m_i} 是特定于模式的复杂性阈值， $\ell_{\max}$ 是最大可容忍边缘利用率， β 是带宽限制。

在多模式任务中，不同的模式通常表现出不同程度的复杂性和资源需求。我们的策略不是对整个输入强制执行统一的卸载决策，而是采用按模式的路由机制来实现部分卸载和细粒度调度。例如，在图像和文本查询中，具有高计算和带宽要求的图像可以被

卸载到云端，同时在边缘本地处理随附的短文本以立即响应。形式上，决策向量定义为

$$d = \pi(c_1, c_2, \dots, c_k, s) \in \text{edge}, \text{cloud}^k, \quad (6)$$

其中 k 表示输入中的模式数量， c_i 是模式 i 的复杂性得分， s 表示系统状态（包括边缘负载和网络带宽）， $\pi(\cdot)$ 是将模式感知阈值与系统级动态集成的自适应调度策略函数。

4. 实验

4.1. 实验装置

我们使用 NVIDIA A100 (40 GB) GPU 作为云服务器，使用 NVIDIA 3090 (24 GB) 作为边缘设备。Qwen2-VL-2B 模型部署在边缘，而 Qwen-2.5-VL-7B 模型部署在云端[18]。评估数据集是 VQAv2[19] 和 MMBench[20]，我们从验证集中随机选择 5000 张图像进行测试。网络带宽分别设置为 200 Mbps、300 Mbps 和 400 Mbps。图像复杂度和文本复杂度的权重设置为它们的平均值，复杂度阈值设置为 0.5。基准方法包括 (1) *Cloud-only* (Qwen-2.5-VL-7B)、(2) *Edge-only* (Qwen2-VL-2B) 和 (3) *PerLLM* (边云协作解决方案)。评估指标包括准确性、端到端延迟和资源开销。

4.2. 主要结果与分析

4.2.1. Accuracy Comparison

如表 1 所示，所提出的 MoA-Off 框架实现了极具竞争力的性能。至关重要的是，它获得了与纯云方法相当的整体准确性，并且性能损失仅为 <0.4%，可以忽略不计。此外，我们的方法在绝对精度方面大大优于仅边缘和 PerLLM，超过 4.8%-16.8%。这些结果凸显了拟议的 MoA-Off 框架的有效性。

表 1. 准确度 (%) 比较结果。

Method	Cloud-only	Edge-only	PerLLM	MoA-Off
<i>VQAv2 Dataset</i>				
200 (Mbps)	76.3	61.4	71.3	76.1
300 (Mbps)	77.4	63.2	71.8	77.2
400 (Mbps)	77.8	63.5	72.4	77.5
<i>MMBench Dataset</i>				
200 (Mbps)	75.6	58.4	68.3	75.2
300 (Mbps)	76.1	60.1	69.2	75.9
400 (Mbps)	76.5	61.2	69.9	76.3

4.2.2. End-to-End Latency Comparison

如图 3 所示，综合延迟测量表明，我们提出的 MoA-Off 框架在所有评估的策略中实现了最低的平均端到端延迟。它的性能明显优于纯云方法和纯边缘方法超过 50%。与其他边云协作方法相比，MoA-Off 的延迟降低了 30% 以上。总的来说，通过智能地将最复杂的推理卸载到云端，我们的方法避免了仅边缘模型在处理困难样本时典型的严重延迟尾部，从而实现了最佳平衡。

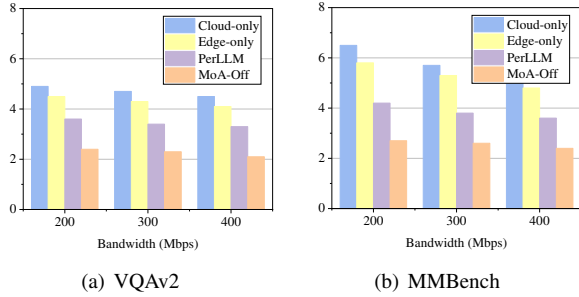


图3.不同方法的端到端延迟比较结果。(a) 显示了 VQAv2 数据集上的结果，而 (b) 显示了 MMBench 数据集上的结果。

4.2.3. Resource Overhead Comparison

计算开销。如图 4 (a) 和图 4 (b) 所示，MoA-Off 在 VQAv2 数据集和 MMBench 数据集上都表现出最低的计算开销。它仅卸载最复杂的样本，从而大大减少了云基础设施的计算负担。与纯云解决方案和 PerLLM 相比，MoA-Off 可以减少 30%-65% 的计算开销。我们的方法在计算模态感知卸载策略时在边缘设备上引入的计算开销可以忽略不计，这比运行 MLMM 轻几个数量级。这种工作负载的战略分配可确保以最少的计算投资解决绝大多数请求，无论是在边缘执行简单任务，还是在云上执行复杂任务。

内存开销。如图 4 (c) 和图 4 (d) 所示，通过引入自适应模态感知卸载，MoA-Off 最大限度地减少了不必要的数据重复和内存保留，有效平衡了边缘和云之间的分配。仅云解决方案受益于丰富的资源，但由于集中式多模式模型执行和频繁的上下文重新加载而产生大量内存成本。相比之下，PerLLM 通过在层次结构中分配推理来减轻部分负担，但其缺乏模态感知调度会导致两个方面的冗余内存消耗

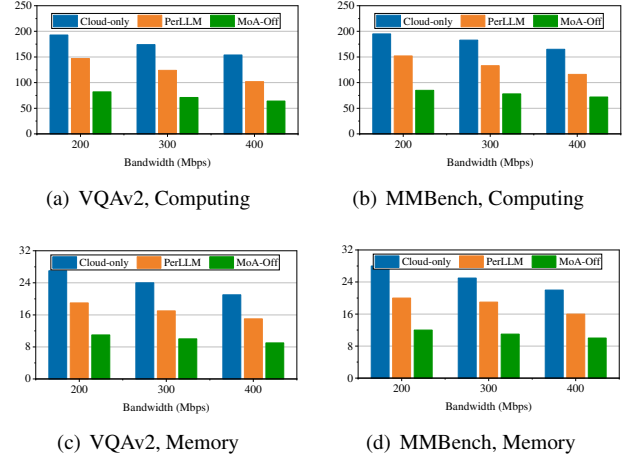


图4.不同方法的资源开销比较结果。(a)和(b)代表计算开销，而(c)和(d)代表内存开销。

结束，特别是在处理具有不同复杂性的异构模式时。

4.3. 消融研究

为了进一步验证我们设计的有效性，我们通过删除自适应模态感知卸载和边缘云协作调度组件来进行消融研究。如果没有模态感知卸载机制，推理精度平均下降 6.8%，反映出系统无法考虑异构模态复杂性。同样，禁用协作调度策略会使端到端延迟增加 21.5%，而由于边缘和云端的任务分配效率低下，计算和内存开销分别增加 18.7% 和 16.3%。这些结果清楚地表明，自适应模态感知卸载和协作调度对于资源受限的边缘云环境下的高效 MLLM 推理是不可或缺的。

5. 结论

在这项工作中，我们提出了 MoA-Off，一种用于高效 MLLM 推理的自适应异构模态感知卸载框架。通过引入轻量级模态感知模块和自适应卸载策略，MoA-Off 有效捕获不同模态的异构特征，并在边缘和云资源之间动态分配工作负载。实验结果表明，所提出的框架在保持有竞争力的推理精度的同时，显著降低了延迟和资源开销，优于传统的仅边缘、仅云和边缘云卸载方法。

6. 参考文献

- [1] Jiayang Wu、Wensheng Gan、Zefeng Chen、Shi Cheng Wan 和 Philip S Yu, “多模态大语言模型: 一项调查”, 2023 *IEEE International Conference on Big Data*, 2023 年, 第 2247-2256 页。[2] Jing Yu Koh、Daniel Fried 和 Russ R Salakhutdinov, “使用多模态语言模型生成图像”, *Advances in Neural Information Processing Systems*, 卷。36, 第21487-21506页, 2023年。[3] Zhihang Lin, Mingbao Lin, Luxi Lin 和 Rongrong Ji, “通过视觉标记撤回促进多模态大语言模型以进行快速推理”, *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025 年, 第36卷, 第21487-21506页, 2023年。39, 第 53 34–5342 页。[4] 陈云凯, 王启萌, 吴世伟, 高岩, 徐童, 胡耀, “Tomgpt: 可靠的纯文本训练方法, 用于经济有效的多模态大语言模型”, *ACM Transactions on Knowledge Discovery from Data*, 卷。18、没有。7, 第 1-19 页, 2024 年。[5] Wenlun 张、Haoran Pang、Yucui Zhou、Shixiao Wang 和 Luking Li, “Gsmm: 资源意识型多模态模型的高效全局稀疏化”, 2025 *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2025 年, 第 1-5 页。[6] 姚远, 于天宇, 张敖, 王崇义, 崔俊波, 朱红基, 蔡天池, 陈驰, 李浩宇, 赵伟林, 等, “用于边缘设备部署的高效 gpt-4v 级多模态大语言模型”, *Nature Communications*, 第 1 卷。16、没有。1, 第 55 09 页, 2025。[7] 郑悦, 陈宇豪, 钱斌, 史秀芳, 舒元超, 陈吉明, “边缘大语言模型综述: 设计、执行和应用”, *ACM Computing Surveys*, 卷。57、没有。8, 第 1-35 页, 2025 年。[8] Zheming Yang、Yunqing Hu、Sheng Sun 和 Wen Ji, “Ec2moe: 自适应端云管道协作实现可扩展的专家混合推理”, *arXiv preprint arXiv:2508.06024*, 2025 年。[9] 王冠群、刘黎明、李晨轩、张元、马俊鹏、魏新宇、张凯文、Maurice Chong、Renrui 张、Yi Jiang Liu 等人, “多模态大语言模型的云设备协作学习”, *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024 年, 第 12646-12655 页。
- [10] Liangqi Yuan、Dong-Jun Han、Shiqiang Wang 和 Christopher G Brinton, “多模式、多任务、多对话设置中的 LLMS 本地云推理卸载”, *arXiv preprint arXiv:2502.11007*, 2025 年。
- [11] Zheming Yang、Yuanhao Yang、Chang Zhao、QiGuo、Wenkai He 和 Wen Ji, “个性化推理调度, 针对多种 llm 服务的边云协作”, *arXiv preprint arXiv:2405.14636*, 2024。[12] Jingjing Xie、Yuxin 张、Mingbao Lin、Liujuan Cao 和 Rongrong Ji, “利用边缘云协作推进多模态大语言模型量化感知尺度学习以实现高效适应”, 载于 *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024 年, 第 10582–10591 页。[13] Zixuan Yi, Zijun Long, Iadh Ounis, Craig Macdonald 和 Richard Mccreadie, “增强推荐系统: 与大型多模态编码器的深度模态对齐”, *ACM Transactions on Recommender Systems*, 卷。3、没有。4, pp. 1–25, 2025。[14] 尹树康, 付超友, 赵思睿, 李科, 孙兴, 徐同, 陈恩宏, “多模态大语言模型综述”, *National Science Review*, 卷。11、没有。12, 第 403 页, 2024 年。[15] 梁童、李勇和高伟, “移动计算的分层边缘云架构”, 载于 *35th Annual IEEE International Conference on Computer Communications*, 2016 年, 第 1-9 页。[16] Zheming Yang、Wen Ji、Qi Guo 和 zhi Wang, “Javp: 用于 dnn 推理的边云协作的联合感知视频处理”, *Proceedings of the 31st ACM International Conference on Multimedia*, 2023 年, 第 9152-9160 页。[17] Henry Bae, Aghyad Deeb, Alex Fleury, and Kehang Zhu, “Complexitynet: 通过学习任务复杂性提高 llm 推理效率”, *arXiv preprint arXiv:2312.11511*, 2023。[18] Jinze Bai, Shuai Bai, Yunnanfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang 等人, “Qwen Technical 报告”, *arXiv preprint arXiv:2309.16609*, 2023 年。[19] Yash Goyal、Tejas Khot、Douglas Summers-Stay、Dhruv Batra 和 Devi Parikh, “让 vqa 中的 v 变得重要: 提升图像理解在视觉问答中的作用”, 载于 *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017 年, 第 17 页。6904–6913。[20] 刘源, 段浩东, 张元涵, 李博, 张松阳, 赵王博, 袁一科, 王佳琪, 何从辉, 刘紫薇等, “Mmbench: 你的多模态模型是全能型选手吗?”, *European Conference on Computer Vision*, 2024 年, 第 216-233 页。